

PREGUNTAS PLANTEADAS Y RESPUESTAS.

1. Calculo de la funcion de costo y variables

La funcion de costo se calculo para minimizar dos cosas: el error de seguimiento de la ruta y el esfuerzo de los motores. Las variables utilizadas fueron el estado del vehiculo (posicion x , posicion y , y el angulo de orientacion θ) y las variables de control (velocidad lineal v y velocidad angular w). Se asignaron pesos mediante matrices llamadas Q y R . La matriz Q penaliza que el robot se aleje de la mariposa, mientras que la matriz R penaliza movimientos muy bruscos de los motores para evitar que el robot patine o consuma la bateria demasiado rapido.

2. Simulacion del sistema mpc

El sistema se simulo integramente en Python antes de tocar el hardware. Se utilizo un modelo cinematico del robot diferencial para predecir como se moveria ante diferentes comandos. En la simulacion se probaron mas de 10 configuraciones distintas hasta encontrar la que lograba cerrar la figura de la mariposa con un error minimo. Esto permitio verificar que el controlador era estable antes de arriesgar los componentes fisicos.

3. Restricciones para el funcionamiento

Las restricciones aplicadas fueron limites fisicos reales. Se restringio la velocidad lineal v a un maximo de 0.4 metros por segundo y la velocidad angular w para evitar giros excesivamente rapidos que el hardware no podria replicar. Tambien se consideraron restricciones de tiempo, usando un intervalo de muestreo de 0.1 segundos, que es el tiempo que tarda el sistema en procesar cada nuevo comando.

4. Consumo del vehiculo y fuente de poder

El vehiculo utiliza un voltaje nominal de 6 voltios proveniente de 4 baterias AA en serie. El consumo de corriente (amperaje) se estima en un pico de 1 amperio cuando ambos motores arrancan simultaneamente, mientras que el ESP32 consume cerca de 200 miliamperios. Esto da una potencia total aproximada de entre 4 y 6 watts en los momentos de mayor esfuerzo. La fuente de poder se diseño unificando las tierras (GND) del circuito de potencia y de control en la protoboard para asegurar que las señales de control fueran claras.

5. Hardware utilizado y justificacion

Se utilizo un ESP32 modelo WROOM. Se escogio por sobre un Arduino convencional porque el MPC genera una cantidad de datos muy grande (2152 puntos) que requieren mucha memoria flash y una velocidad de procesamiento alta (240 MHz). Un Arduino comun no habria tenido memoria suficiente para almacenar la trayectoria completa ni la velocidad de reloj para procesar la cinematica inversa con la precision necesaria. No se uso Raspberry Pi o Jetson Nano porque habrian sido excesivas en costo y consumo energetico para un robot de este tamaño.

6.Codigo y librerias utilizadas

Se utilizaron dos lenguajes. Python se uso para la etapa de optimizacion y simulacion del MPC debido a sus potentes librerias de calculo numerico. Para el vehiculo real se uso C++ mediante el entorno de Arduino IDE. No se usaron librerias externas complejas en el ESP32 para maximizar la velocidad; se utilizo la libreria estandar de matematicas y las funciones nativas del ESP32 para el control de señales PWM (ledc).

7. La mariposa y sus dimensiones

La mariposa se genero mediante ecuaciones parametricas. Originalmente la trayectoria era muy grande para interiores, por lo que se tuvo que redimensionar usando un factor de escala de 0.23. Esto ajusto la figura a un area aproximada de 2 por 2 metros. El paso a coordenadas cartesianas se hizo en Python, convirtiendo cada punto de la curva en una coordenada x e y que luego el MPC utilizo como referencia para generar los comandos de velocidad.

8. Division del proyecto + 9. Participacion del equipo

Todos hicimos de todo. Pero nos distribuimos así:

SOFTWARE: Jorge, Ignacio (MPC, Simulación, ESP32)

HARDWARE: Franco, José y Jorge x2 (Electrónica, componentes, potencia, etc).

10. Armado y materiales del vehiculo

El vehiculo se armo sobre un chasis de acrilico. Se usaron dos motores DC de piñoneria plastica (motores amarillos) y un puente H modelo L298N para el control de potencia. No se usaron sensores externos (como encoders) para este proyecto, confiando en el control de lazo abierto calculado por el MPC. En los datasheet se encontro que los motores operan idealmente entre 3 y 6 voltios y que tienen una corriente de estancamiento (stall) que podria quemar el driver si no se limitaba la potencia en el codigo.

11. Dificultades y soluciones

La principal dificultad fue la zona muerta de los motores y la asimetría; un motor era más débil que el otro. Esto se solucionó mediante software creando un piso de potencia mínimo (MIN PWM) y un multiplicador extra (BOOST) para el motor más lento. Otra dificultad fue que el robot hacía curvas muy abiertas, lo que se solucionó usando un ancho de vía virtual en el código, engañando al sistema para que forzara giros más cerrados.

12. Otros aspectos y bitacora de ajustes

Se mantuvo una bitacora de más de 15 intentos de calibración. Un aspecto clave fue la reducción de la frecuencia de los motores a 500 Hz o menos para ganar torque. También fue vital el uso de un botón de inicio (BOOT) para evitar que el robot saliera disparado antes de colocarlo en el suelo. El ajuste final de la constante de velocidad máxima fue lo que permitió que la mariposa teórica de Python se pareciera finalmente a la trayectoria real en el patio.